

Predicting Home Runs and Animal Shelter Outcomes

By: Robert Masci

Abstract

Machine learning, a subset of Artificial Intelligence in the field of Computer Science, analyzes algorithms whose performance improves as they are exposed to more data over time. This project will specifically compare machine learning models such as Decision Trees, K-Nearest Neighbors, Logistic Regression, Support Vector Machines, RandomForests, and Artificial Neural Networks across two distinct datasets. Through hyperparameter tuning and performance analysis, this study examines how different architectures handle binary and multiclass datasets. The results highlight the strengths and limitations particular models have with different datasets, the impact of class imbalance on metric selection, and the large impact that we as developers have when it comes to model selection, tuning, and evaluation.

Introduction

This project aims to deepen our understanding of the machine learning process, from exploring binary and multiclass datasets, selecting and tuning models, and comparing and analyzing performance. Four machine learning models were tuned and trained on a binary classification dataset that involved baseball hit outcomes with the goal of predicting whether or not the hit was a home run. I selected Decision Tree, K-Nearest Neighbors (KNN), Logistic Regression, and Support Vector Machines (SVM) models to attempt to solve this goal. These algorithms each have their own processes for learning patterns from the data, such as Decision Tree dividing data points based on thresholds in their feature values. As for another example, KNN works by identifying the most common label among the K nearest neighbors to a datapoint to predict the current points label, so I thought perhaps similarly batted balls could potentially have similar outcomes and the model could succeed. Logistic Regression and SVM are a notable comparison, as both are rooted in linear decision boundaries, but SVM adds a layer of complexity by utilizing kernel functions to handle non-linear relationships and find the maximal margin between classes. Four additional models were tuned and trained on a multiclass classification dataset that involved animal shelter outcomes with the goal of determining if the animal ended up adopted, transferred, returned to their owner, euthanized, or died. For this dataset, I chose Decision Tree, KNN, RandomForest, and Artificial Neural Network. Artificial Neural Networks utilize hidden layers of nodes to learn non-linear relationships in the data, so I thought the added complexity of the model might give it a better chance at succeeding. Also, a comparison to highlight is between Decision Tree and RandomForest models, where RandomForest is an ensemble method that utilizes many Decision Tree models trained using bootstrap aggregating in an effort to reduce the overall variance for improved performance. This project begins by discussing the datasets and reviewing the methodology for preprocessing the data. Then, I go over the hyperparameter tuning process with the goal of optimizing model performance for the given data. Finally, I will provide the final results produced by the optimal models on the test datasets in the form of performance metrics and confusion matrices and conclude with a comparative analysis of the model performances.

Datasets

My binary classification home run dataset was posted to Kaggle as a competition. The data is batting outcomes collected from games during the 2020 MLB season. The target variable that I am trying to predict is whether or not a batting outcome resulted in a home run. There are 25 columns in total in the train.csv, including the target variable "is_home_run." After one hot encoding, the transformations, and dropped columns, I ended with 19 features. The most important features would appear to be the launch_speed and launch_angle, as they are clearly the strongest predictors of a home run. How hard the ball comes off the bat and the angle it comes off would certainly have the biggest effect.

The train.csv comes with 46,244 rows prior to any pruning. There were a significant amount of missing values specifically for the launch_speed and launch_angle where there were 11,805 (25.53%) and 11,785 (25.48%) missing values respectively. I dropped the rows with the missing values, which left me with 25,866 rows of data to work with. It was a tough decision because the launch metrics are likely the most predictive, but removing them could impart bias if there was a reason the data was not recorded properly, such as weak contact not producing capturable data. However, I felt the benefit outweighed the risk. The data was split with scikit-learn's train_test_split method to make a testing dataset of 20% of the dataset. I stratified it according to the target variable "is_home_run," to ensure the testing dataset would have a representative sample because the data shows just 5.03% of the rows being the positive class versus 94.97% being the negative class after pruning, which is very imbalanced.

The main challenge with the dataset is the highly imbalanced classes, where the positive class is only 5.03%. An imbalance of this level will make it difficult to achieve success at predicting the positive class, but I will take measures to try to maximize success including using class_weight balanced where possible and tuning based on F1 score to try to maximize the balance between precision and recall.

My multiclass dataset has to do with the outcomes of animals in an animal shelter. The dataset comes from Kaggle where it is titled Shelter Animal Outcomes. This data comes from Austin Animal Center and was collected from 10/01/2013 to 03/2016. The target variable that I am trying to predict is the outcome of the animal from the shelter. There are 5 different classes for this target variable being Adoption, Transfer, Return_to_owner, Euthanasia, and Died. The train.csv file's shape revealed 26,729 rows, which was more than enough to work with and split a testing set off of. To split this dataset I used scikit-learn's train_test_split method to make a testing dataset of 20% of the original dataset, and I stratified it according to the target variable "OutcomeType," to ensure the testing dataset would have a representative sample of the 5 different classes.

I ended with 9 features after processing, which included dropping AnimalID, DateTime, and OutcomeSubtype, as well as transforming some features into more usable formats. The name column in the training dataset has a lot of missing data (6,174 values), but this is actually a contributing feature itself because the animals having or not having a name could influence their outcome. Other than that, there was not a significant amount of missing values in the other features, so those 19 rows were simply dropped. The target variable percentage distribution in the training split is 40.32/35.21/17.92/5.81/0.74 with 8,615 samples belonging to the most common class (Adoption) and 158 to the most rare (Died). This is an interesting split as it's heavily balanced to the top 2 and the last class has a very small portion. The class imbalance with the Died class being so small at 0.74% and Euthanasia being 5.81% could complicate model training as they are rare. It's very important to tune the models to pick up on Euthanasia, as perhaps the model could be used to intervene on the animals that are most likely to be put to sleep and try to improve their outcomes.

Methods

For preprocessing my baseball data, I dropped a few of the columns that are not predictive, such as `bip_id`, `batter_id`, `pitcher_id`, `batter_name`, `pitcher_name`, `park`, and `game_date`. I also engineered a few new features such as `batter_is_home` (using `batter_team` and `home_team`) and `same_handedness` (using `is_batter_lefty` and `is_pitcher_lefty`). I transformed `pitch_name` into a single binary classifier of `is_fastball`, as one-hot encoding the several different pitch types would add significant dimensionality that is more costly than necessary. In baseball terms, the main categorization of pitch types is fastballs versus offspeed pitches. I one-hot encoded `bb_type` and `bearing`. I dropped the rows with missing `launch_speed`/`launch_angle` values as they are likely the most predictive features and keeping that 25% of rows could significantly affect model performance. It is possible this introduced bias if the lack of launch data was related to certain outcomes, but it's a worthwhile tradeoff in my opinion. This left me with 25,866 rows of data to work with. I normalized my data for the KNN, Logistic Regression, SVM models as they are distance sensitive, and some of my data like `plate_x` are small values between -2 and 2 while the `launch_speeds` are 7.7 through around 118 and `launch_angle` between -87 and 90. To address the class imbalance, I used scikit-learn's `class_weight='balanced'` parameter for Decision Tree, Logistic Regression, and SVM, which adjusts the class weights inversely proportional to their frequency, giving more importance to the minority class during training. KNN does not support this parameter so it was left without this adjustment. I used stratified k-fold cross validation with 5 folds. The stratification was necessary to ensure the validation set has the proper class balance. I have created tuning curves for `max_depth` for Decision Tree, `K` for KNN, `C` for Logistic Regression, and `C & gamma` for SVM. I used grid search for the remaining hyperparameters, being the criterion for Decision Tree, weights for KNN, and solver for Logistic Regression. I used scikit-learn's `GridSearchCV`, which trains and evaluates models with each combination of the categorical hyperparameter values, and used the resulting best model as the model to use on my test dataset.

I used F1 score as the scoring parameter because the default of accuracy will not be ideal for my imbalanced dataset. I chose F1 score because it balances a model's precision and recall, so it makes sure that it finds a good middle ground between never predicting positive and just guessing positive too often. The other metric is Precision-Recall AUC. I switched to this as opposed to the ROC-AUC because that could have been misleadingly optimistic on my heavily imbalanced dataset, as it would produce good scores just for identifying the majority class well. PR-AUC instead looks at only the positive class and basically ignores the true negatives entirely so it will more accurately identify how well the models perform at identifying home runs. Additionally, I created confusion matrices for all of my models to visually see how well they performed at maximizing true positives and minimizing false positives.

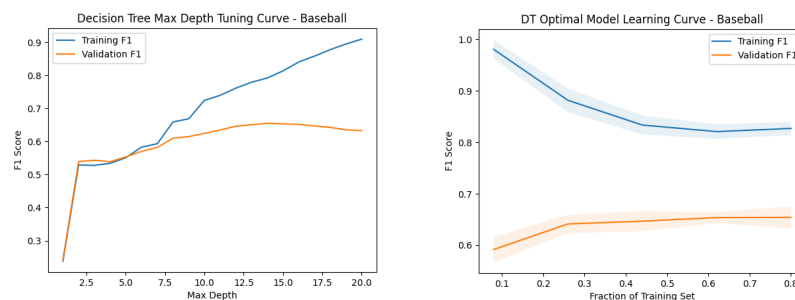
For preprocessing my animal dataset, I dropped `Color`, `DateTime`, `AnimalID`, and `OutcomeSubtype` as they are either not predictive or would increase dimensionality far beyond their predictive value. I transformed the `name` feature into `has_name`, a binary feature where if the entry is null it will be no, otherwise yes, and similarly `AnimalType` is a simple binary `is_dog`. I transformed the `Ageuponoutcome` from the strings like "2 years" to `age_in_days`. I had the idea to transform the `SexUponOutcome` feature to two binary features `is_neutered` and `is_male` instead of one-hot encoding into the 4 different variations. However, this would have required me to drop 864 rows (4.04%) where `SexUponOutcome` was input as `Unknown` and it could introduce bias as `unknown` corresponded to 87.9% transfer and 8.9% euthanized compared to the full sample being 35.2% and 5.8% respectively. So, I

instead one-hot encoded SexUponOutcome into all 5 categories including Unknown. I also transformed the breed from the many different possibilities to a is_purebred feature where if there are multiple breeds listed, or mix in the name it is no, otherwise yes. I normalized age_days for both KNN and ANN as that number could range from zero to thousands while many of my other features are just binary features and that would skew results. To address the class imbalance, I used scikit-learn's class_weight='balanced' parameter for Decision Tree and RandomForest. KNN and ANN do not support this parameter so they were left without this adjustment. Again, I used stratified k-fold cross validation with 5 folds as it balances training on a sizable portion of training data and being more computationally efficient. The stratification is necessary for this dataset as well to ensure the validation set has the proper class balance and doesn't just leave out the 0.74% death outcome. I have a tuning curve for max_depth for Decision Tree, K for KNN, max_depth and n_estimators for RandomForest, and alpha for ANN. I used grid search for the remaining hyperparameters, being the criterion for Decision Tree, distance metric for KNN, and hidden_layer_sizes for ANN. Again, I've used scikit-learn's GridSearchCV, making sure to use macro F1 score as the scoring parameter because accuracy score will not be ideal for my imbalanced dataset. I took the resulting best model from the GridSearch to use on my test dataset.

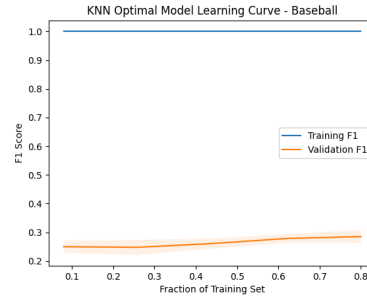
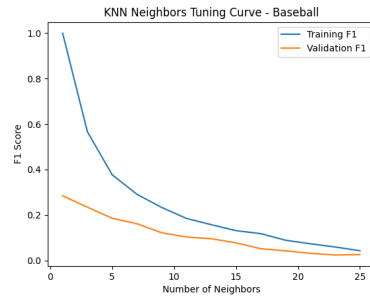
The first metric I used to assess model performance was macro F1 score because of the imbalance in the dataset. The macro version of F1 score calculates the F1 score for each class and then takes an unweighted average. The other metric was weighted F1 score because I thought it would be interesting to compare the two versions of F1. With these two scores, if a model scored well on the weighted F1 but lower on the macro F1 we would know it performed worse on the rarer classes and put less importance on identifying them. I additionally have created confusion matrices for all of the models to visualize and better assess their performance.

Hyperparameter Tuning and Other Analyses

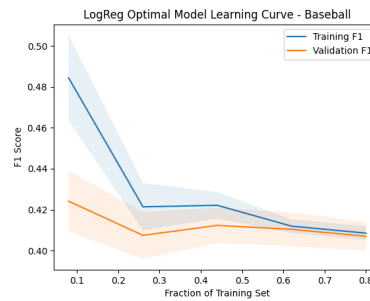
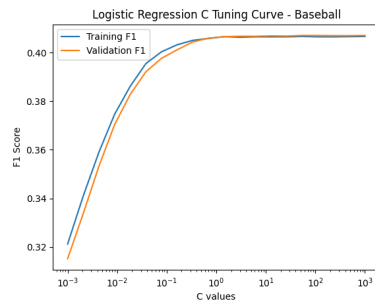
Home Run Dataset



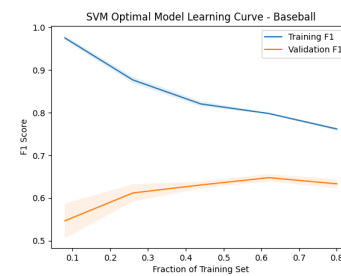
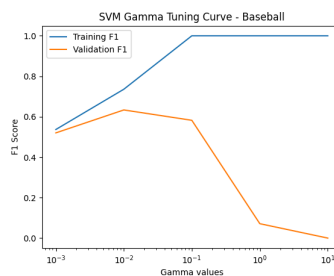
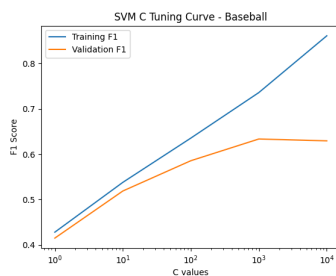
Above is the hyperparameter tuning curve for max_depth and the learning curve for the Decision Tree model on the baseball dataset. The optimal depth appears to occur at around 14 as the validation F1 score peaks there. GridSearch determined the optimal splitting criterion to be Gini over Entropy and Log Loss, which is interesting as Entropy is generally the more sensitive criterion, but is much more computationally expensive. It seems the difference in increased complexity offered by Entropy did not produce better results for this dataset. The learning curve shows a rather wide gap between the training and validation, which suggests the model is somewhat overfit to the data. Despite this, the model was the optimal configuration determined by GridSearch, so all other configurations must have produced worse F1 scores on the unseen validation data.



Above is the tuning curve for neighbors and the learning curve for the KNN model on the baseball dataset. The optimal neighbors is found at just 1, which is severely overfit, but apparently more neighbors just lowers the F1 score more even when doing k-fold cross validation. GridSearch determined the optimal weighting to be uniform over distance. As the model found the optimal K to be 1, it appears the weighting did not play a particularly big role. The learning curve shows a massive overfitting as the optimal model of 1 neighbor means it just memorizes the training data.



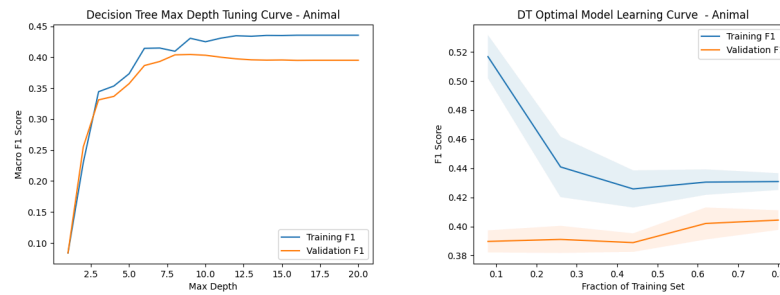
The figures above display the hyperparameter tuning curve for the regularization strength (C) and the resulting learning curve for the Logistic Regression model. The optimal C continuously increases and then plateaus beginning around 1, though GridSearch determined 1000.0 to be optimal, so it must continue climbing. This preference for a very high C indicates that the model performed best when regularization was minimized, allowing the coefficients to fit the training data as closely as possible without slack. GridSearch determined the optimal solver to be lbfgs over liblinear and saga. The learning curve shows both training and validation being close at a low point on the curve meaning the model is likely underfit. This isn't necessarily surprising as logistic regression produces a hyperplane to try to separate data but if it is not linearly separable the model would struggle even with adding more data.



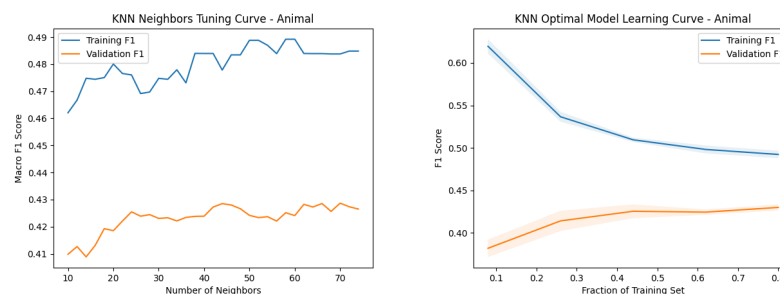
Above are the hyperparameter tuning curves for regularization strength (C) and for Gamma, as well as the learning curve for the SVM model on the baseball dataset. The optimal C appears to peak at around 1000, which is similar to Logistic Regression and suggests that the model found the best results in

minimizing the amount of slack permitted. The optimal Gamma appears to peak around 0.01. Gamma controls the influence radius of a single datapoint, and the value of 0.01 is relatively low, which actually means that the points have far reaching influence. This results in a smoother, more linear decision boundary and generally higher bias, or underfitting. The learning curve shows a gap between the training and validation that is a sign of overfitting, so perhaps the low gamma was an attempt to curb this.

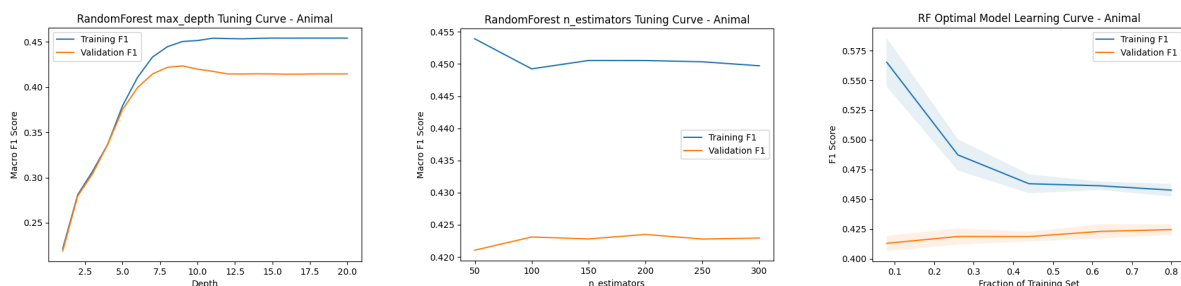
Animal Shelter Dataset



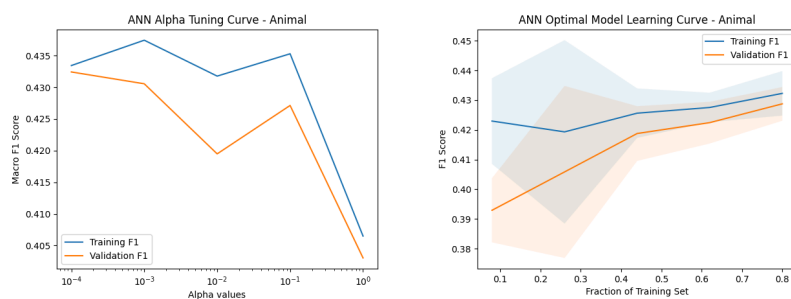
Above are the hyperparameter tuning curve for max_depth and the learning curve for the Decision Tree model on the animal dataset. The GridSearch found the optimal max_depth to be 9, which is around where the graph appears to peak at. GridSearch determined the optimal splitting criterion to be entropy over gini and log loss, so a different finding than it had on the baseball dataset. The learning curve shows both training and validation being close at a relatively low point on the curve meaning the model is likely underfit, which is an interesting finding as the max_depth being 9 is relatively deep. Perhaps the Decision Tree as a model is not very well suited to finding patterns in this dataset.



Above are the hyperparameter tuning curve for neighbors and the learning curve for the KNN model on the animal dataset. GridSearchCV identified the optimal neighbor count to be 45, with distance-based weighting performing better than uniform weighting. This combination of hyperparameters suggests that while 45 neighbors are considered for every point, the closer data points have more weight, as that is the significance of the distance-based weighting. With that being said however, the learning curve shows a relatively small gap between training and validation low on the F1 score, which suggests the model is a bit underfit and not scoring particularly well in F1. Again it doesn't appear to have found good patterns in the dataset to learn it very well.



Above are the hyperparameter tuning curves for max_depth and n_estimators, as well as the learning curve for the RandomForest model on the animal dataset. The optimal max_depth appears to be 9, which was the same in Decision Trees. The optimal n_estimators is 200. By training 200 different trees on various bootstrap samples of the data, RandomForest attempts to reduce variance in single trees. However, the learning curve for Decision Tree showed an underfit pattern rather than overfit. Here again, the learning curve again shows a relatively small gap between training and validation low on the F1 score, which suggests an underfit model as it does not even score well on the training dataset.



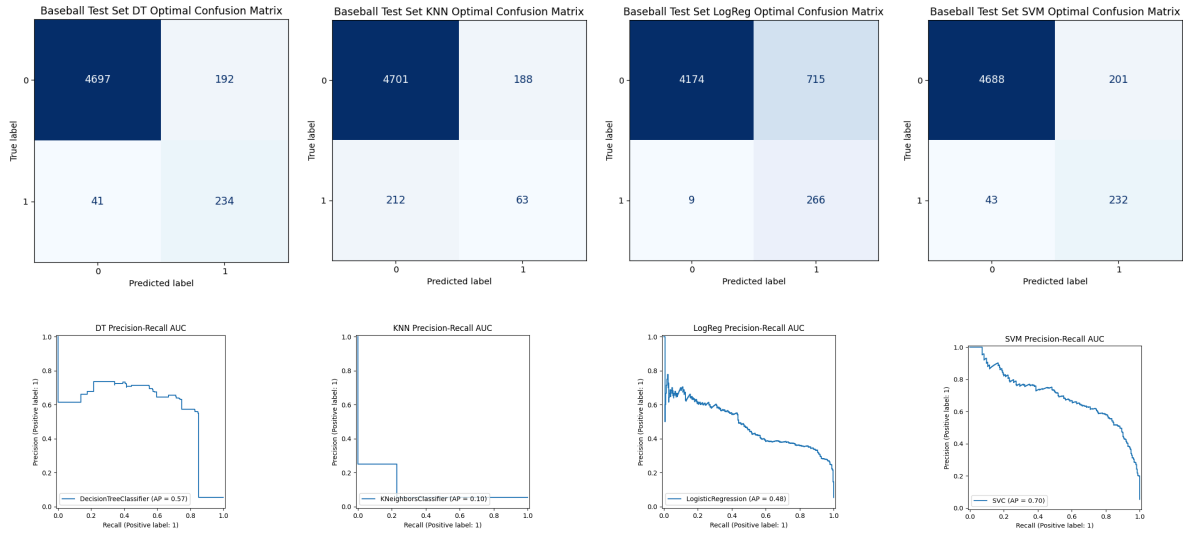
Above are the hyperparameter tuning curves for alpha and the learning curve for the ANN model on the animal dataset. The optimal alpha appears to be around 0.0001. I provided GridSearch with the hidden_layer_size options (50,50), (100,100), (100,50,50), (100,100,100), (50, 100, 50), and (50, 50, 50, 50, 50, 50). It determined the optimal hidden layer size to be (100, 100, 100). The learning curve again shows a relatively small gap between training and validation low on the F1 score, which suggests an underfit model as it does not even score well on the training dataset.

Results

Home Run Dataset

Baseball Dataset Result

Model	Train F1	Val F1	Test F1	Test PR-AUC
Decision Tree	0.7918	0.6549	0.6676	0.5750
KNN	1.0000	0.2847	0.2395	0.0986
Logistic Regression	0.4066	0.4070	0.4236	0.4833
SVM	0.7359	0.6335	0.6554	0.6966

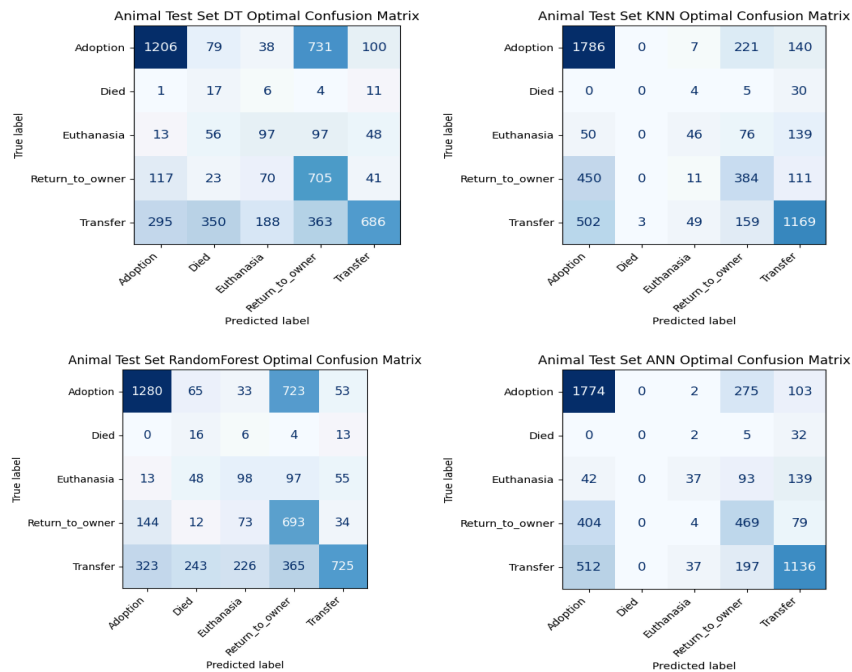


The results above display the F1 scores that the optimal models determined by GridSearch achieved on the training, validation, and test datasets, as well as the PR-AUC scores on the test dataset. Additionally, there are the 4 confusion matrices and 4 PR-AUC charts.

Animal Shelter Dataset

Animal Dataset Result

Model	Train Macro F1	Validation Macro F1	Test Macro F1	Test Weighted F1
Decision Tree	0.4306	0.4044	0.3920	0.5362
KNN	0.4834	0.4299	0.4075	0.6176
RandomForest	0.4505	0.4235	0.4011	0.5519
ANN	0.4334	0.4324	0.4117	0.6254



The results above display the macro F1 scores that the optimal models determined by GridSearch achieved on the training, validation, and test datasets, as well as the weighted F1 scores on the test set. Additionally, there are the 4 confusion matrices on the test datasets.

Discussion

The home run prediction results above tell a very interesting story together. The validation F1 scores suggested that Decision Tree and SVM would be far superior to Logistic Regression and KNN, the latter of which being very poor. This was reflected in the other metrics as well, as the first two I mentioned were much higher in the testing F1 and PR-AUC scores that ran on the test dataset. We can also see how F1 alone does not tell the full story of the models performance because, while it informs us about the models ability to balance precision and recall, it doesn't always mean the highest F1 score will be the best for identifying the positive class. This is seen by the confusion matrices and PR-AUC scores and charts which focus on the positive class. Based on Test F1 and even in the confusion matrices Decision Tree appears the best where it identifies 2 more home runs and makes 9 less false positive predictions. However, SVM's PR-AUC score is significantly higher at 0.6966 versus 0.5362, which takes into account the performance across different thresholds whereas the confusion matrix is just at 0.5. So, basically if we had set a lower threshold value for predicting home runs, such as 0.3 as opposed to 0.5, SVM would have done a better job at predicting home runs correctly than the Decision Tree model. Both of these models blow away the other two however, as Logistic Regression does worse, but KNN is seemingly horribly suited for the dataset. It resorted to using neighbors of 1 and basically just memorized the training dataset and consequently did horribly on the other sets. However, this was the best model on unseen validation data, so apparently increasing neighbors would have only produced even worse results. Logistic Regression was underfit as evidenced by the learning curve and failed to learn the data well, leading to a poor performance in the F1, confusion matrix, and PR-AUC. However, it apparently did very well in terms of recall as the confusion matrix shows it got 266 of the 275 actual home runs predicted, but it was not precise at all with 715 false positives, much higher than the others.

As for the animal shelter outcomes results, they unfortunately show that all four models struggled with the dataset as their macro F1 scores on the test set all hover around 0.4. There is a bigger discrepancy in their weighted F1 scores, where KNN and ANN saw the biggest gains over their macro scores. As mentioned in the Methods section, this could suggest that they performed better on the majority classes and were less capable of identifying the rare classes of euthanasia and death. This is confirmed by the confusion matrices where we can see that KNN got 0 of the 39 deaths correct and only 46 of the 311 euthanasias correct. Similarly, ANN got 0 deaths correct and only 37 euthanasias correct. So these models basically ignored the rare classes to maximize identifying the more common classes where they correctly identified significantly more adoptions and transfers than Decision Tree and RandomForest. On the other hand, Decision Tree identified 17 deaths and 97 euthanasias. This is balanced by misidentifying 731 adoptions as Return to owner, but this is not as harmful as misidentifying the euthanasias where we would want to intervene and help the animals. In terms of false positives for euthanasia, there are 302 false positives which is the most of the four models. RandomForest performed similarly, where it identified 16 deaths, 98 euthanasias, and 338 false positives for euthanasia. Their improved minority class performance can perhaps be attributed to Decision Tree and RandomForest being able to utilize the `class_weight` balanced attribute, which gives inverse weight to a class based on

its relative size. Another interesting finding is that perhaps RandomForest did not excel here because it, like other ensemble methods using bootstrap aggregating, generally succeeds when averaging out high variance models, but the Decision Tree model appeared to be high bias rather than high variance. If I had the foresight to predict that the models would generally underfit the data and therefore have high bias, I might have been better off choosing an ensemble method that utilized boosting rather than bagging in order to reduce high bias. That being said, none of these models performed particularly well on the dataset as evidenced by the low F1 scores and spread out confusion matrices.

Conclusion

This project detailed the machine learning pipeline process from dataset selection and analysis, through model tuning, and finished with model performance analysis. We saw how models like SVM and Decision Tree were better suited towards the imbalanced home run prediction binary dataset than were KNN and Logistic Regression. We also saw how all the models failed to adequately learn the animal shelter outcome dataset despite our best efforts at tuning. Though even in this failure, we saw the impacts that parameters like `class_weight` being set to `balanced` can impact model performance, as well as the importance of model selection in a bootstrap aggregating ensemble being chosen over a boosting one. Finally, across both datasets we saw the importance of metric selection, as our model performances could be interpreted drastically differently based on just looking at F1 or PR-AUC for the baseball set or just looking at macro F1 versus weighted F1 for the animal set. Only when we look at all the metrics, as well as the confusion matrices, does the full picture of model performance come together in a more comprehensive picture. With more time and foresight, I would have liked to examine how a boosting ensemble method, such as `AdaBoostClassifier` or `GradientBoostingClassifier`, would have handled the animal shelter dataset, as it could have tried reducing the bias that the models tended to have. Additionally, I would have liked to pursue more advanced feature engineering for the animal dataset because the preprocessing that I did seemingly did not do enough to mitigate bias in the models. Perhaps alternative processing to the breed feature or reexamination of the color feature could have aided model learning. Finally, future exploration could integrate SMOTE, or Synthetic Minority Over-Sampling Technique) to place more importance on addressing class imbalance.

References

1. GeeksforGeeks. "Gamma Parameter in SVM." GeeksforGeeks, 10 June 2024, www.geeksforgeeks.org/machine-learning/gamma-parameter-in-svm/.
2. GeeksforGeeks. "Macro Average vs Weighted Average." GeeksforGeeks, 9 June 2025, www.geeksforgeeks.org/machine-learning/macro-average-vs-weighted-average/.
3. GeeksforGeeks. "PrecisionRecall Curve ML." GeeksforGeeks, 19 July 2019, www.geeksforgeeks.org/machine-learning/precision-recall-curve-ml/.
4. GeeksforGeeks. "SMOTE for Imbalanced Classification with Python." GeeksforGeeks, 3 May 2024, www.geeksforgeeks.org/machine-learning/smote-for-imbalanced-classification-with-python/.
5. Kraggle. "Baseball." Kaggle.com, 2020, www.kaggle.com/datasets/jcraggy/baseball/.
6. "Shelter Animal Outcomes." @Kaggle, 2026, www.kaggle.com/competitions/shelter-animal-outcomes/.